

• WORKSHOP • LEVEL 1 • 60 MINUTES

FinOps Basic.

Build a working Claude plugin for financial services — in Markdown, with no code. Skills, slash commands, and your first sub-agent in a single hour.

WHAT YOU'LL BUILD

```
finops-basic/  
├── .claude-plugin/  
│   └── plugin.json  
├── skills/  
│   ├── compliance-language.md  
│   └── client-meeting-prep.md  
├── commands/  
│   ├── prep-meeting.md  
│   ├── draft-report.md  
│   └── review-draft.md  
└── agents/  
    └── compliance-reviewer.md
```

7 MODULES 40 SLIDES 0 LINES OF CODE



Agentic • Automations

LAB GUIDE • BUILDING CLAUDE PLUGINS FOR
FINANCIAL SERVICES

• WHAT YOU'LL BUILD

A plugin that turns Claude into a financial specialist.

FinOps Basic — a folder of Markdown files that Claude loads as one cohesive plugin. Four moving parts, each doing a single job well.

SKILL

Compliance language

Fires automatically on every draft — keeps disclaimers and risk language in by default.

COMMAND

/prep-meeting

Generates a full client briefing pack from a name and a few notes.

COMMAND

/draft-report

Turns a paragraph of quarter performance into a polished, compliant client report.

SUB-AGENT

compliance-reviewer

Audits drafts before they leave your desk. Independent. Auditable. Hands back a structured report.

• TODAY'S SESSION • 60 MIN

Seven modules.

1.1 Plugin structure

The folder. The three primitives. The golden rule.

1.3 Test the sample

Load it. Three real prompts.

1.5 Test your customisation

Reload, run, verify.

1.7 Package and share

Zip it. Send it. Done.

1.2 Explore the sample

Read every file. Two discussion prompts.

1.4 Customise

Pick one exercise: skill • command • rules.

1.6 Sub-agents

Specialist Clauses you can delegate to.

THEN →

Level 2 • MCP & orchestration

• WHO THIS IS FOR

Anyone who uses Claude daily.

Business analysts, financial advisers, operations staff. No prior plugin work assumed — if you can edit a text file, you can finish this lab.

- ✓ You build & test using only Markdown and a text editor
- ✓ You leave with a working plugin you can hand to a colleague
- ✓ You finish in one hour

• WHAT YOU NEED

Three things.

01 A paid Claude subscription

Pro, Max, Team, or Enterprise.

02 A text editor

VS Code, Notepad++, nano — whatever you already use.

03 The workshop files

`plugin-files.zip` — unzip it anywhere.

1.1

• MODULE ONE

Plugin structure.

A Claude plugin is a folder with a specific layout. Once you know the layout, the rest of the lab is fill-in-the-blanks.

• THE FOLDER

Four folders. One JSON. The whole thing.

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json      # what this plugin is
├── skills/
│   └── my-skill.md      # auto-fires on relevant work
├── commands/
│   └── my-command.md    # trigger with /command-name
└── agents/
    └── my-agent.md      # specialist sub-Claudes
```

- **plugin.json** — the identity card. Name, version, description.
- **skills/** — background knowledge Claude applies automatically.
- **commands/** — slash commands you trigger by name.
- **agents/** — focused specialists, invoked by commands or skills.

— NOTE

There is no compilation step. Edit a Markdown file, reload the plugin, you're done.

• THE GOLDEN RULE

When each one fires.

Skills

Fire **automatically** when relevant. The user doesn't know they're there.

Commands

Fire when you type **/command-name**. Explicit, visible, deliberate.

Agents

Are **invoked** by commands or skills to handle a focused task.

1.2

• MODULE TWO

Explore the sample.

Open the `level1-finops-basic` folder. Read every file. Then we'll discuss two questions.

• SEVEN FILES TO READ

The sample plugin, end-to-end.

01 `.claude-plugin/plugin.json`
The plugin's identity card

02 `skills/compliance-language.md`
Rules Claude follows on every output

03 `skills/client-meeting-prep.md`
The meeting prep workflow

04 `commands/prep-meeting.md`
Slash command — briefing pack

05 `commands/draft-report.md`
Slash command — client report

06 `commands/review-draft.md`
Slash command — delegates to the agent

07 `agents/compliance-reviewer.md`
The sub-agent that reviews drafts

→ 5 minutes
Skim each one — we'll discuss next

• DISCUSSION • 01

What if you removed the compliance skill?

WHAT CHANGES

- weekly-summary, draft-report and prep-meeting produce output without disclaimers — unless the prompt template hard-codes them itself.
- /review-draft still catches problems — but only if someone remembers to run it.
- Claude's training gives it generic awareness of financial compliance, but not *your firm's* exact disclosure text.

THE REAL POINT

— DEFAULT - SAFE

The skill makes compliance the path of least resistance. Without it, compliance becomes opt-in — depending on whoever ran */review-draft* or remembered to ask for disclaimers.

In a regulated context, that's the difference between **guardrails** and **guidelines**. The plugin doesn't break without the skill — responsibility just shifts from the system to the user.

• DISCUSSION • 02

Why is **prep-meeting** a command, not a skill?

01

It needs a target

The command takes \$ARGUMENTS — the client name or pasted notes. The user must supply context, so the user must initiate it.

02

It produces a deliverable

The output *is* the point — a formatted briefing pack. That's a document the user explicitly requests, not a background behaviour.

03

As a skill, it'd be chaos

Imagine briefing packs firing every time a client's name was mentioned. Skills should be invisible. Commands should be visible.

— AND THE INVERSE

Compliance-language has to be a skill — you don't decide to write compliantly, you always should. Forcing users to run `/compliance-language` before every draft would defeat the purpose entirely.

• THE DESIGN RULE • REMEMBER THIS

Commands “Do this thing now, with this input.”

Skills “Always behave this way,
without being asked.”

1.3

• MODULE THREE

Test the sample.

Load the plugin, then run three prompts. Watch how each piece fires — automatic, on-demand, and delegated.

• TWO WAYS TO LOAD IT

Cowork or Claude Code.

MACOS • WINDOWS

Option A • Cowork UI

- Claude Desktop → Cowork tab → Plugins
- Click **Upload Plugin** → select the folder
- Start a new Cowork session

NOTE

Windows arm64 is not supported by Cowork — use Claude Code instead.

ALL PLATFORMS • INCL. LINUX

Option B • Claude Code

```
# install once
npm install -g @anthropic-ai/claude-code

# launch with the plugin loaded
claude --plugin-dir ~/plugin-level1/finops-basic
```

RECOMMENDED FOR BUILDING

Faster iteration loop than the Cowork upload flow.

• TEST 01 • THE SKILL FIRES AUTOMATICALLY

Does it correct the "definitely"?

PASTE THIS INTO CLAUDE

Write a short email to a client telling them their portfolio returned 12% last year and will **definitely** return 15% next year.

— WATCH FOR

Claude refuses or rephrases "definitely." Past performance disclaimer is appended. No forward-looking guarantees survive.

WHY IT WORKS

No `/command` was needed. The compliance skill fired because the topic matched its description.

WHAT THE SKILL ENFORCES

- No guarantees on future returns
- Past performance qualifier
- Standard firm disclaimer text

• TEST 02 • SLASH COMMAND

/prep-meeting — a full briefing pack.

RUN THIS

```
/finops-basic:prep-meeting Sarah Chen — retail investor, $250K portfolio, moderate risk, focused on ASX blue chips and some international exposure. Last meeting we discussed rebalancing away from banks.
```

Claude produces a structured pack — context recap, agenda, talking points, suggested questions — with compliance language already woven in by the skill.

WHAT YOU'LL SEE

- ✓ Client snapshot reconstructed from your notes
- ✓ Talking points framed neutrally
- ✓ Suggested rebalance discussion
- ✓ Disclaimer wording appears without being asked

• TEST 03 • SLASH COMMAND

/draft-report — polished, compliant.

RUN THIS

/finops-basic:draft-report Q4 2024: Portfolio up 8.2% vs ASX 200 up 6.1%.
Top performers: BHP +18%, CSL +12%. Underperformers: TLS -4%. Cash position
at 8%, slightly elevated.

— COMPARE

Same numbers, but Claude outputs a properly structured client report — performance overview, attribution, positioning, disclaimer. Five minutes of formatting work done in three seconds.

THREE THINGS TO NOTICE

- ✓ Numbers framed against the benchmark
- ✓ Attribution split out per holding
- ✓ Compliance language sits at the bottom — every time

1.4

• MODULE FOUR • 15 MIN HANDS-ON

Customise.

Pick **one** of three exercises. Make the plugin yours. Then we'll reload and test in 1.5.

• PICK ONE

Three ways to extend the plugin.

A

Add a new skill

Teach Claude how emails at your organisation should sound — tone, greetings, sign-offs, words to avoid.

AUTO-FIRES

B

Add a new command

Build `/weekly-summary` — turn a week of client logs into one structured team report.

ON-DEMAND

C

Customise compliance

Edit the existing skill — your firm's actual disclaimer, product-specific rules, ASIC wording.

EDIT EXISTING

• EXERCISE A • NEW SKILL

Make a folder. Make a SKILL.md.

FILE LOCATION

```
skills/
├── email-tone/
│   └── SKILL.md
```

FRONTMATTER FORMAT

```
---
name: email-tone
description: Guides the tone and structure
  of all client emails. Apply automatically
  when drafting any client email.
user-invocable: false
---

# Email Tone Guidelines
...your rules here...
```

WHAT TO PUT IN IT

- ✓ Preferred greeting style
- ✓ Tone guidance — formal vs conversational
- ✓ How to close emails
- ✓ Phrases your team uses or avoids

— WHY USER-INVOCABLE: FALSE

Skills run in the background. The user shouldn't have to ask for "good email tone" — it should just happen.

• EXERCISE B • NEW COMMAND

A flat `.md` file becomes a command.

FILE LOCATION

```
commands/
└─ weekly-summary.md
```

THE FRONTMATTER

```
---
description: Summarise a week of client
  interactions into a structured report.
argument-hint: [paste interaction notes]
---
```

...your instructions here...

THEN INVOKE IT

```
/finops-basic:weekly-summary [paste your client interaction logs
here]
```

— TIP • SEED YOUR DATA FIRST

Ask Claude to generate a folder of realistic client logs before testing. Then pass that folder path as the argument.

The filename becomes the command name. Lowercase, hyphenated — no other config required.

• EXERCISE C • EDIT EXISTING

Make compliance match your firm.

EDIT THIS FILE

skills/compliance-language/SKILL.md

THREE THINGS WORTH ADDING

- ✓ Your organisation's actual disclaimer text — paste it in verbatim
- ✓ Product-specific language rules (managed funds vs SMSF vs direct equities)
- ✓ Region-specific wording — ASIC for AU, FCA for UK, SEC for US

— WHY THIS MATTERS MOST

The sample disclaimer is generic. Your firm's lawyers wrote exact wording — paste it here and every output the plugin produces will use it.

No new files. No new structure. Just a tighter, firm-specific version of what already exists. The fastest win of the three exercises.

• PRO TIP

Let Claude write the skill.

Open a **second terminal**, run `claude` (no plugin loaded). Describe what you want. Paste the output into the right file.

— WHY A SECOND SESSION

You're using Claude to write Claude config. Keeping it separate from the plugin session keeps the context clean and lets you iterate quickly.

- Describe behaviour in plain English
- Ask Claude to write the file end-to-end
- Copy into `skills/<name>/SKILL.md`
- Reload the plugin, test, iterate

TERMINAL • PLUGIN SESSION

```
$ claude --plugin-dir ./finops-basic
Plugin loaded • 2 skills • 3 commands • 1 agent
> /prep-meeting Sarah Chen...
```

↓ SEPARATE WINDOW

TERMINAL • AUTHOR SESSION

```
$ claude
> Write a SKILL.md that enforces our email tone...
Drafting skills/email-tone/SKILL.md...
```

SAME TRICK WORKS FOR COMMANDS TOO

Just point Claude at `commands/<name>.md` instead.

• PASTE THIS • THEN FILL IN YOUR SPECIFICS

A ready-to-use prompt.

```
I'm building a Claude Code plugin for a financial services firm.
I need a new skill file at:
  skills/email-tone/SKILL.md

The skill should fire automatically whenever Claude drafts a client email.
It should NOT be user-invocable (background knowledge only).

Our email tone guidelines are:
- Always open with "Dear [First Name],"
- Conversational but professional – no jargon
- Close with "Kind regards, [Your Name]"
- Never use urgency language like "act now" or "don't miss out"
- Always include our standard disclaimer at the bottom

Please write the complete SKILL.md file including correct frontmatter.
```

THEN TEST SILENTLY

Reload the plugin. Ask Claude to draft an email — **without** mentioning tone at all.

```
Draft an email to James telling him his portfolio is
up 6% this quarter.
```

The skill fires in the background — greeting, tone, sign-off, disclaimer all in place. The user never asked.

1.5

• MODULE FIVE

Reload and verify.

A plugin is just files on disk. To pick up your changes, you exit the session and reload — that's the whole loop.

• THE RELOAD LOOP

Two commands. One question.

CLAUDE CODE • ALL PLATFORMS

Exit, then relaunch

```
claude --plugin-dir ~/plugin-level1/finops-basic
```

Same flag you used the first time. The plugin re-reads every file on launch.

COWORK • MACOS • WINDOWS

Refresh the plugin

- Plugins panel → your plugin → refresh / re-upload
- Start a fresh Cowork session

Cowork caches the plugin per session — a fresh session is the cleanest way to verify.

— THE QUESTION

Did the new skill or command behave the way you intended? If not — go back, refine the description, reload, try again. The whole loop is under a minute.

1.6

• MODULE SIX

Sub-agents.

Specialist Claudes your plugin can delegate to. Hire a compliance officer — don't ask your generalist assistant to also be one.

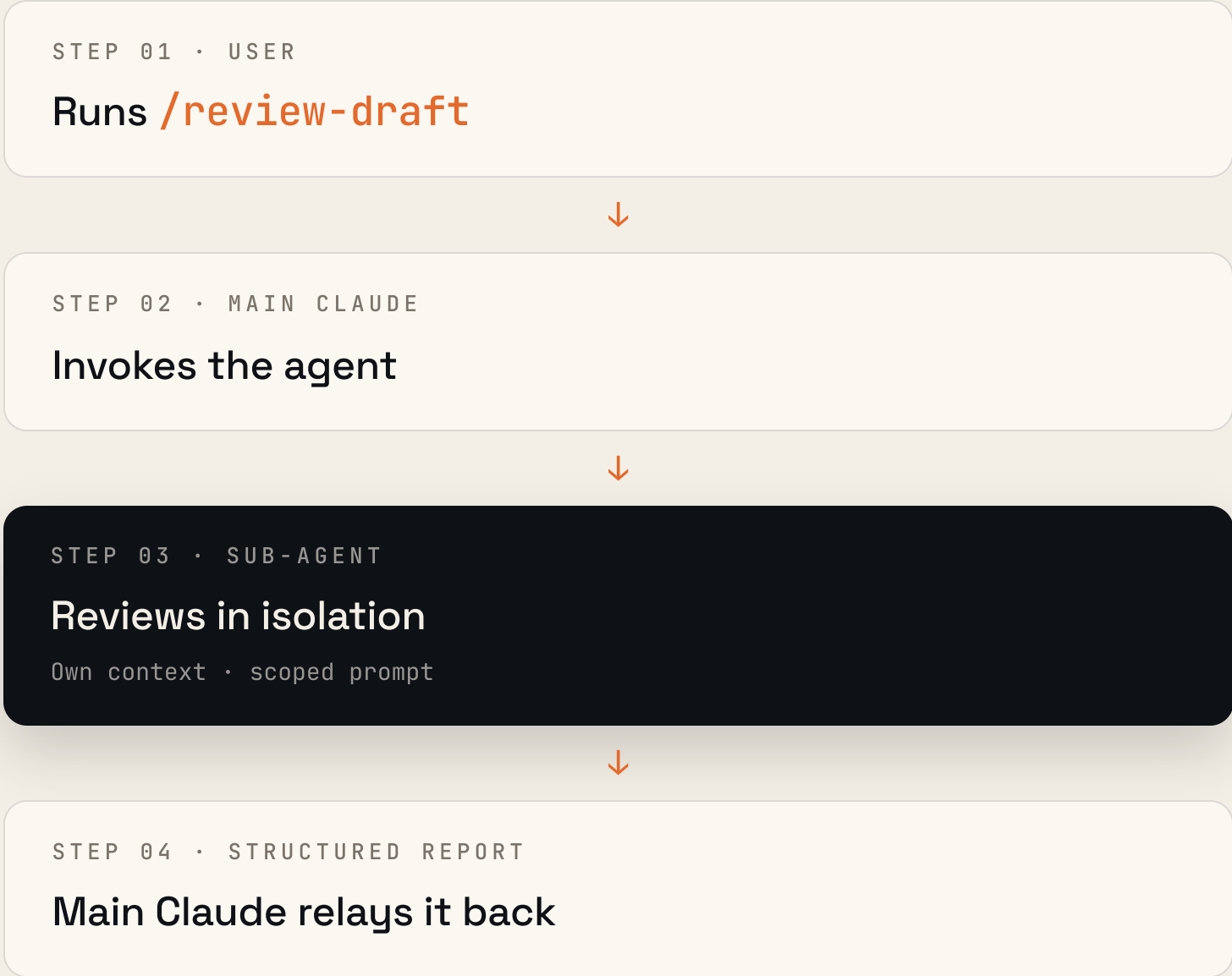
• THE MENTAL MODEL

Hire a specialist.

Instead of asking your generalist assistant to also be your compliance officer — spin up a dedicated reviewer who does only that job. Thoroughly. Consistently. Every time.

— HOW IT FLOWS

A command (or skill) invokes the agent → the agent runs in its own focused context → it returns a structured report → the main Claude weaves it into the user-facing response.



• WHY SUB-AGENTS

Four reasons.

01

Focused

Tightly-scoped system prompt — the agent can't drift into other topics.

02

Composable

Any command or skill can invoke them on demand.

03

Structured output

Rigid format means the parent can parse and re-use it reliably.

04

Auditable

Predictable workflows you can review, log, and trust over time.

• THE AGENT FILE

Where it lives, what's inside.

FILE LOCATION

```
my-plugin/
├── .claude-plugin/plugin.json
├── skills/
├── commands/
└── agents/
    └── compliance-reviewer.md
```

OPEN THE SAMPLE

```
agents/compliance-reviewer.md
```

THREE THINGS TO NOTICE

- ✓ YAML frontmatter includes `model:` — you specify which Claude model handles this agent
- ✓ System prompt is **narrow** — it only reviews, never drafts
- ✓ Output format is **rigidly defined** — the parent can parse the response reliably

— THE CONSTRAINT IS THE VALUE

A general-purpose Claude is endlessly capable. A scoped sub-agent is endlessly reliable. Pick the right one for the job.

• TEST • THE REVIEWER IN ACTION

Hand it a deliberately bad draft.

RUN THIS

`/finops-basic:review-draft`

Dear Sarah,

Your portfolio has had an **incredible** year and will **definitely** continue to outperform the market.
I **guarantee** you'll see at least **12%** returns next year based on our **best-in-class** strategy.

Regards,
James

WHAT THE AGENT CATCHES

- ✓ **"definitely"** — forward-looking guarantee
- ✓ **"guarantee"** — explicit promise of returns
- ✓ **"12% returns next year"** — specific forecast
- ✓ **"best-in-class"** — unsubstantiated superlative
- ✓ Missing past-performance disclaimer

The agent returns a structured report. The main Claude turns that into a plain-English summary back to you.

• DISCUSSION

Why a separate agent — instead of asking the main Claude to also check?

Asking the main Claude to self-review is like asking a lawyer to proofread their own contract.

Technically possible. Practically unreliable. A separate agent with a single role, a fixed checklist, and no authorial attachment is **structurally incapable** of the biases that make self-review weak.

— THE PRINCIPLE

Separation of concerns isn't a software pattern. It's how you build trust into a system that has to be right every time.

• HANDS-ON • BUILD YOUR OWN

A meeting-summariser agent.

CREATE

```
agents/
└─ meeting-summariser.md

commands/
└─ summarise-meeting.md
```

THE AGENT'S CONTRACT

- ✓ Single role — summarise meeting notes only
- ✓ Exact output: agenda recap · decisions made · action items (owner + deadline)
- ✓ Invoked by the new `/summarise-meeting` command

RELOAD, THEN TEST

```
/finops-basic:summarise-meeting Sarah Chen — retail investor,
$250K portfolio, moderate risk, focused on ASX blue chips and
some international exposure. Last meeting we discussed
rebalancing away from banks.
```

— WATCH FOR

A clean three-section response — every time, in the same format. That predictability is exactly what a sub-agent buys you.

1.7

• MODULE SEVEN • LAST ONE

Package and share.

Your plugin is just a folder. Zip it. Send it. Your colleagues run the same one-line load command.

• ONE ZIP • TWO WAYS IN

Zip it.

MACOS • LINUX

From your shell

```
cd plugin-files
zip -r finops-basic-myplugin.zip \
    level1-finops-basic/
```

WINDOWS • POWERSHELL

One-liner

```
Compress-Archive `
    -Path level1-finops-basic `
    -DestinationPath finops-basic-myplugin.zip
```

— HOW COLLEAGUES LOAD IT

macOS / Windows — Cowork → Plugins → Upload Plugin • **Linux** — `claude --plugin-dir ./level1-finops-basic`

• WHAT YOU BUILT

Level 1, recap.

1

PLUGIN • FOLDER OF MARKDOWN

2

SKILLS • AUTO-FIRE IN BACKGROUND

3+

COMMANDS • SLASH-TRIGGERED

1

SUB-AGENT • SCOPED SPECIALIST

0

LINES OF CODE WRITTEN

60m

IDEA TO PACKAGED PLUGIN

• WHEN YOU'RE READY

Level 2 picks up here.

01

MCP tools

Wire your plugin into external systems your firm already runs.

02

Live market data

Pull real prices, fundamentals, and news straight into your prompts.

03

Bash pipeline

A real data layer — your plugin can now read, transform, and write.

04

Multi-agent orchestration

Several specialists working in concert — coordinated by one command.

• TAKE THIS WITH YOU

Resources.

WORKSHOP MATERIALS

EDA-repository/finops-claude-plugins

All Level 1 plugin files, the lab guide, and the Level 2 starter on github.com.

ANTHROPIC EXAMPLES

anthropics/knowledge-work-plugins

Official patterns for skills, commands and sub-agents — useful for inspiration.

GET HELP

Workshop Slack channel

Post screenshots of your plugin output — fastest way to get unstuck.

ISSUES & FEEDBACK

GitHub issues

Found a bug in the sample plugin? Raise it on the workshop repo.

Thank you.

You now have a working plugin, the design rules to extend it, and a clean path into Level 2 whenever you're ready.

01 / 07

